Reviewer Pack — Minimal Order of Quaternionic Kähler Metrics and Circuit Mon…

Entry 65a1a52a95f6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 15:21:30 UTC

Minimal Order of Quaternionic Kähler Metrics and Circuit Monotone Width Inequality

Entry ID: `65a1a52a95f6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 15:21:30 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Geometry (Quaternionic Kähler Metrics) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every Boolean circuit with monotone width W, there exists a corresponding Quaternionic Kähler metric on the projective space of complex lines such that the dimension of the moduli space of Kähler metrics at the critical point is upper-bounded by $\Theta(W^2)$.

Rationale (proposer's reasoning):

> Quaternionic Kähler geometry offers a rich algebraic-geometric framework for studying the geometric properties of complex structures. If there were a direct relationship between these metrics and circuit monotone width, it could potentially lead to new insights into circuit complexity lower bounds.

Taxonomy category: `QuaternionicGeometryCircuitComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6864da63df0cd98c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each Boolean circuit with monotone width W, the dimension of the moduli space of Kähler metrics at the critical point is upper-bounded by $\Theta(W^2)$ if and only if the mean dimension across 30 random seeds is $\leq W^2$.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quaternionic Kähler Metrics" AND "circuit monotone width inequality" - "projective space complex lines" AND "moduli space dimension upper bound" - "Boolean circuit complexity" AND "Quaternionic Kähler metric"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def generate_boolean_circuit(W):
    if W == 1:
        return ['0', '1']
    w = random.randint(1, W-1)
    left = generate_boolean_circuit(w)
    right = generate_boolean_circuit(W-w)
    return [f'({l} & {r})' for l in left] + [f'({l} | {r})' for l in right]

def evaluate_circuit(circuit):
    variables = set()
    for expr in circuit:
        if '&' in expr or '|' in expr:
            variables.update(expr.split()[1:-1])
    values = {var: random.choice(['0', '1']) for var in variables}
    stack = []
    for expr in circuit[::-1]:
        if expr == '0' or expr == '1':
            stack.append(expr)
        elif '&' in expr:
            left = stack.pop()
            right = stack.pop()
            stack.append('1' if left == '1' and right == '1' else '0')
        elif '|' in expr:
            left = stack.pop()
            right = stack.pop()
            stack.append('0' if left == '0' and right == '0' else '1')
    return stack[0]

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
    results = []
    for W in [5, 10, 15, 20, 30, 40]:
        instances_tested = 0
        n_max = W
        total_dimension = 0
        for _ in range(30):
            circuit = generate_boolean_circuit(W)
            dimension = evaluate_circuit(circuit)  # Simplified for testing purposes
            results.append((W, dimension))
            instances_tested += 1
        mean_dimension = sum(dimension for _, dimension in results) / len(results)
        conjecture_holds = mean_dimension <= W**2
        counterexample = "" if conjecture_holds else f"Mean dimension: {mean_dimension}, Expected:
        return {
            "metric_name": "Moduli Space Dimension",
            "metric_value": mean_dimension,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value)**2 for result in results) / len(results
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='Mean dimension exceeds expected' first_failing_s
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e00905.py", line 74, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e00905.py", line 54, in run_trial
    circuit = generate_boolean_circuit(W)
```

```
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e00905.py", line 22, in generate_bool
    left = generate_boolean_circuit(w)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e00905.py", line 22, in generate_bool
    left = generate_boolean_circuit(w)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e00905.py", line 22, in generate_bool
    left = generate_boolean_circuit(w)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f2e00905.py", line 24, in generate_bool
    return [f'({l} & {r})' for l in left] + [f'({l} | {r})' for l in right]
                      ^
NameError: name 'r' is not defined. Did you mean: 're'?
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's claim. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14143
2	preregistration	ollama_remote	glm4:latest	0	0	9066
3	novelty	ollama_remote	glm4:latest	0	0	8162
4	novelty	ollama_remote	glm4:latest	0	0	12036
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25691
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8171
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7748
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10008
9	judge	ollama_remote	glm4:latest	0	0	16863

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 111888 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/65a1a52a95f6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/65a1a52a95f6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/65a1a52a95f6.tar.gz` (if generated)